



AutoML Motivation

Chapter Summary

Marius Lindauer





Overview: Chapter 1 – Introduction to AutoML

This chapter covered the following topics:

1. **Why is ML not easy?** – The challenges motivating AutoML
2. **What is AutoML?** – Definition and key advantages
3. **Formal Definition** – The AutoML optimization problem
4. **Success Stories** – AutoML in competitions and industry
5. **When to use AutoML?** – Guidelines for practitioners
6. **Interfaces to AutoML** – How to interact with AutoML tools
7. **Components of AutoML** – Tasks, approaches, and post-processing



1. Why is ML not Easy?

“Machine learning is the science of getting computers to act without being explicitly programmed.”
(Andrew Ng)

In practice, ML workflows require many manual decisions:

- **Hyperparameters** – learning rate, batch size, regularization, LoRA rank, ...
- **Model/Architecture** – model class (XGBoost vs. DNN), network design, operators
- **Data preprocessing** – normalization, augmentation, filtering

Wrong choices → failure. Right choices → great success.

~> AutoML is the solution to these problems!



2. What is AutoML?

AutoML contributes to the *efficient, reproducible, and systematic* design of effective intelligent systems by creating a general automated development process, targeting the full ML workflow.

Key advantages:

 **Efficiency & Effectiveness** – Outperforms human teams in data science competitions (e.g., AutoGluon: Top 3 in 17/21 Kaggle competitions in 2024, each with ~ 1500 human teams)

 **Consistent Systematics** – Structured search strategies (grid, random, Bayesian Optimization) eliminate human bias

 **Reproducibility** – Required by the EU AI Act and academic standards; software reruns identically, humans do not



3. Formal Definition of AutoML

Standard ML task: learn a model $\hat{f}$ minimising empirical risk: $\hat{f} \in \arg \min_{f \in \mathcal{H}} \mathcal{R}_{\text{emp}}(f, \mathcal{D})$

AutoML task: find the optimal configuration λ^* of an inducer $\mathcal{I}$:

$$\lambda^* \in \arg \min_{\lambda \in \Lambda} \mathcal{R}_{\text{gen}}(\mathcal{I}(\mathcal{D}_{\text{train}}, \lambda), \mathcal{D}_{\text{val}})$$

The **configuration space** Λ can describe:

- **Hyperparameters** – learning rate, data augmentation strength, ...
- **Neural architecture** – operators, connections, cell structure
- **ML pipeline** – preprocessing steps + model selection
- **Ensemble** – which models to combine

Note: Proper validation schemes are crucial to avoid overfitting to the validation set.



4. Success Stories

Competitions:

- **ChaLearn AutoML Challenge (2015)** – First full AutoML competition; key ingredients: Bayesian Optimization on full pipelines + ensembling
- **NeurIPS'20 BBO Challenge** – More advanced methods (e.g., BO) consistently beat grid/random search
- **2024 AutoML Grand Prix** – AutoML vs. humans under time pressure; winner: LightAutoML; data preprocessing was decisive

Industry & Research:

- AutoML used by NVIDIA, Intel, Mitsubishi, Oracle, Pinterest, ...
- Applied to non-standard domains (e.g., shape error prediction in milling)
- **Foundation models (2025)** – TabPFN enables “load-and-predict” w/o training or HPO



5. When to Use AutoML?

Do use AutoML when...

- The design/search space is large and hard to navigate
- No good baseline exists (rapid prototyping)
- Good hyperparameter settings are unknown
- Reproducibility and documentation are required
- A measurable fairness constraint exists

Don't use AutoML when...

- Known constraints directly imply a model
- A well-established baseline already exists
- Default settings are known to work (e.g., ImageNet)
- Complex ensembles hurt interpretability
- The relevant fairness metric is unknown

If you already have insights into your problem – use them first!



6. Interfaces to AutoML

AutoML tools can expose different input/output interfaces:

Interface	Input	Output
Data $\rightarrow$ Model	Dataset	Predictive model
Data + Constraints $\rightarrow$ Model	Dataset + time/compute constraints	Model
Data + Space $\rightarrow$ Model	Dataset + config. space	Model
Data + Space $\rightarrow$ Config	Dataset + config. space	Optimal configuration
Data + Docu $\rightarrow$ Code	Dataset + task description	Generated code

The choice depends on the use case:

- Domain experts prefer the simplest interface (data $\rightarrow$ model)
- ML experts benefit from defining the configuration space themselves



7. Components of AutoML

AutoML can target the **full pipeline** or individual components:

Tasks addressed by AutoML:

- Hyperparameter optimization (HPO)
- Neural Architecture Search (NAS)
- Full pipeline design (algorithm selection + HPO)
- Data augmentation & preprocessing

Approaches used in AutoML: Bayesian Optimization, evolutionary methods, reinforcement learning, multi-fidelity methods, ...

↪ *No one-size-fits-all method; choice of AutoML approach has diminishing returns compared to task choice.*

Post-processing: ensembling, stacking, post-hoc explainability



Chapter Summary

Key takeaways from Chapter 1:

1. ML workflows require many manual decisions – AutoML automates these
2. AutoML improves **efficiency**, **systematics**, and **reproducibility**
3. Formally, AutoML minimises the **generalisation risk** over a configuration space Λ
4. AutoML has proven its value in competitions and real-world industry applications
5. AutoML is most useful for large design spaces, unknown baselines, or when reproducibility matters – *but domain knowledge should always be leveraged first*
6. Different **interfaces** exist depending on the user's expertise and goals
7. AutoML encompasses many **tasks** (HPO, NAS, pipelines) and **approaches** (BO, evolutionary, ...), with post-processing steps such as ensembling

⇒ AutoML makes ML more accessible, systematic, and effective for everyone!